

LELBC: caratteristica differenziale e recupero parziale di chiave

Elisa Morganti

Ottobre 2026

1 Introduzione

Il presente progetto ha come obiettivo lo studio pratico della crittoanalisi differenziale applicata a LELBC, un cifrario a blocchi lightweight progettato per dispositivi IoT con risorse limitate. Il lavoro prende come riferimento l'articolo *Differential Cryptanalysis of a Lightweight Block Cipher LELBC*, nel quale vengono studiate le proprietà differenziali del cifrario mediante tecniche MILP, individuando in particolare una caratteristica differenziale a 9 round nel modello single-key con probabilità complessiva pari a 2^{-60} e una caratteristica a 12 round nel modello related-key con la stessa probabilità complessiva. Nel progetto non viene riprodotto il modello MILP del paper, ma si realizza un'analisi sperimentale e implementativa centrata sulla struttura del cifrario, sulla costruzione della Difference Distribution Table della S-box, sulla propagazione delle differenze, sulla verifica empirica delle probabilità e infine su un recupero parziale della sottochiave dell'ultimo round di una versione ridotta di LELBC.

L'obiettivo principale è quindi mostrare in modo concreto il legame tra struttura del cifrario, comportamento differenziale della S-box, costruzione di una caratteristica differenziale, distinguisher e key recovery. L'intero progetto è stato implementato in Mathematica utilizzando operazioni bitwise, liste, associazioni, enumerazione controllata e simulazioni Monte Carlo.

2 Descrizione di LELBC

LELBC è un block cipher che opera su plaintext di 64 bit utilizzando una master key di 128 bit. Il plaintext viene suddiviso in due parole da 32 bit, indicate come X_0 e X_1 , mentre la chiave da 128 bit viene elaborata da un key schedule che genera le sottochiavi dei vari round. Il cifrario esegue 16 round e in ciascun round utilizza i 64 bit più significativi della sottochiave corrente, suddivisi in due parole da 32 bit, K_0 e K_1 .

Il key schedule è stato implementato rappresentando la chiave master come una lista di 128 bit. Ad ogni round vengono eseguite le trasformazioni previste dall'algoritmo: applicazione della S-box ai bit $k_{124}, \dots, k_{127}$, XOR della costante di round con i bit $k_{121}, \dots, k_{124}$, scambio delle due metà da 64 bit e rotazioni circolari delle due metà rispettivamente di 9 e 15 bit. Da questo stato aggiornato vengono poi estratti i due word K_0 e K_1 utilizzati nella funzione di round.

La S-box di LELBC è una permutazione involutoria a 4 bit definita da

$$S = \{C, E, 6, A, 4, F, 2, 7, 9, 8, 3, B, 0, D, 1, 5\}.$$

Essendo una S-box a 4 bit, ogni word da 32 bit viene suddiviso in 8 nibble e la sostituzione viene applicata indipendentemente a ciascuno di essi. Nel codice questa operazione è realizzata dalla funzione `ApplySubLayer`, che converte il word in una lista di 8 cifre esadecimali, applica la S-box a ciascun nibble e ricostruisce infine il valore a 32 bit.

Un singolo round di LELBC opera secondo la sequenza

$$X_0 \leftarrow X_0 \oplus K_0,$$

$$X_1 \leftarrow X_1 \oplus \text{ROL}_5(X_0),$$

$$X_0 \parallel X_1 \leftarrow S(X_0) \parallel S(X_1),$$

$$X_0 \leftarrow X_0 \oplus \text{ROL}_5(X_1),$$

$$X_1 \leftarrow X_1 \oplus K_1,$$

dove ROL_5 indica una rotazione circolare a sinistra di 5 bit. Al termine dell'ultimo round viene effettuato lo swap dei due rami, per cui il ciphertext è dato da

$$Y = X_1 \parallel X_0.$$

Nel progetto sono state implementate sia la cifratura sia la decifratura. La funzione **LELBCRound** realizza il round diretto, **LELBCEncrypt** applica iterativamente i round mediante **Fold**, mentre **LELBCInverseRound** e **LELBCDecrypt** implementano il processo inverso. La decifratura è semplificata dal fatto che la S-box è involutoria, quindi vale

$$S^{-1}(x) = S(x).$$

La correttezza dell'implementazione è stata verificata cifrando un plaintext di prova, decifrando il ciphertext risultante e controllando che il plaintext recuperato coincidesse esattamente con quello iniziale.

3 Difference Distribution Table e probabilità differenziali

La parte centrale della crittoanalisi differenziale riguarda il comportamento delle differenze XOR attraverso la S-box. Considerando due ingressi x e $x' = x \oplus \Delta X$, si definisce la differenza di uscita come

$$\Delta Y = S(x) \oplus S(x \oplus \Delta X).$$

Per ogni possibile differenza di ingresso ΔX e differenza di uscita ΔY è stata costruita la Difference Distribution Table, definita da

$$DDT[\Delta X, \Delta Y] = \# \{x \in \{0, \dots, 15\} : S(x) \oplus S(x \oplus \Delta X) = \Delta Y\}.$$

Poiché la S-box ha 16 possibili ingressi, la probabilità di una transizione differenziale della singola S-box è

$$P(\Delta X \rightarrow \Delta Y) = \frac{DDT[\Delta X, \Delta Y]}{16}.$$

Nel codice questa operazione è implementata dalla funzione

$$\text{SBoxDiffProbability}[\text{dx}, \text{dy}].$$

Per estendere il calcolo a un intero word da 32 bit, il word viene suddiviso in 8 nibble e la probabilità complessiva della sublayer viene calcolata come prodotto delle probabilità delle singole S-box:

$$P_{\text{sublayer}} = \prod_{i=1}^8 P(\Delta X_i \rightarrow \Delta Y_i).$$

Questa operazione è implementata dalla funzione **SubLayerDiffProbability**. L'uso della DDT permette quindi di passare da una descrizione puramente strutturale della propagazione delle differenze a una descrizione probabilistica dell'intera caratteristica differenziale.

4 Caratteristica differenziale

Nel progetto è stata utilizzata la caratteristica differenziale single-key riportata nella Table III del paper. Essa specifica una sequenza di differenze dello stato a 64 bit attraverso i round. La differenza iniziale è

$$(\Delta X_0, \Delta X_1) = (40000000, 0020A008),$$

e la caratteristica prosegue con gli stati

$$(00040000, 00802000),$$

$$(00000000, 00004000),$$

$$(00020000, 00001000),$$

$$(10040000, 00801000),$$

fino alla differenza finale della caratteristica a 9 round.

Per verificare autonomamente la probabilità della caratteristica è stata implementata la funzione `RoundCharacteristicProbability`. Data una differenza di ingresso

$$(\Delta X_0, \Delta X_1),$$

la differenza che entra nella S-box del ramo sinistro è semplicemente

$$\Delta U_0 = \Delta X_0,$$

mentre quella che entra nella S-box del ramo destro è

$$\Delta U_1 = \Delta X_1 \oplus \text{ROL}_5(\Delta X_0).$$

Questo deriva dal fatto che nel modello single-key l'XOR con la stessa round key non modifica la differenza, mentre XOR e rotazioni propagano le differenze in modo deterministico. La componente probabilistica è invece dovuta alle transizioni attraverso le S-box.

Conoscendo inoltre la differenza prevista al round successivo, è possibile ricostruire le differenze di uscita delle due sublayer e calcolare quindi la probabilità della transizione di round. Applicando questo procedimento a tutte le transizioni della caratteristica si ottengono probabilità con esponenti

$$-7, -6, -3, -9, -11, -8, -3, -5, -8.$$

Moltiplicando tutte le probabilità di round si ottiene

$$P_{tot} = 2^{-7} \cdot 2^{-6} \cdot 2^{-3} \cdot 2^{-9} \cdot 2^{-11} \cdot 2^{-8} \cdot 2^{-3} \cdot 2^{-5} \cdot 2^{-8} = 2^{-60},$$

in accordo con il risultato riportato nel paper.

Sono state inoltre calcolate le probabilità cumulative dei prefissi della caratteristica. In particolare, dopo tre round si ottiene

$$P_3 = 2^{-16},$$

dopo quattro round

$$P_4 = 2^{-25},$$

e la probabilità continua a diminuire rapidamente all'aumentare del numero di round. Questo risultato è importante perché permette di stimare la data complexity di un esperimento Monte Carlo: se una caratteristica ha probabilità 2^{-k} , il numero medio di coppie necessario per osservare una right pair è dell'ordine di

$$2^k.$$

Per questo motivo una verifica empirica diretta di caratteristiche molto lunghe diventa rapidamente impraticabile.

5 Verifica empirica della caratteristica

Per verificare sperimentalmente la caratteristica non è sufficiente osservare soltanto la differenza finale del ciphertext, perché più trail differenti possono condurre dalla stessa differenza iniziale alla stessa differenza finale. È quindi necessario osservare anche gli stati intermedi del cifrario. A questo scopo è stata implementata la funzione `LELBCEncryptTrace`, che utilizza `FoldList` per conservare lo stato interno dopo ogni round.

Per una coppia di plaintext P e P' si ottengono quindi due sequenze di stati

$$X^{(1)}, X^{(2)}, \dots$$

e

$$X'^{(1)}, X'^{(2)}, \dots$$

e si calcolano le differenze

$$\Delta X^{(r)} = X^{(r)} \oplus X'^{(r)}.$$

La funzione `EmpiricalCharacteristicTest` genera una sola master key casuale, mantenuta fissa per l'intero esperimento, e costruisce per ogni plaintext casuale P un secondo plaintext

$$P' = P \oplus \Delta P,$$

dove ΔP coincide con la differenza iniziale della caratteristica.

Il codice conta separatamente due tipi di eventi. Il primo è il semplice match della differenza finale:

$$\Delta_{out} = \Delta_{attesa},$$

che misura un differenziale input-output. Il secondo richiede invece che tutte le differenze intermedie coincidano esattamente con quelle della caratteristica:

$$\Delta X^{(1)} = \Delta_1, \quad \Delta X^{(2)} = \Delta_2, \quad \dots$$

Questo secondo conteggio misura realmente la probabilità della specifica caratteristica differenziale. Nel test sui primi tre round è stato utilizzato un milione di coppie. Poiché la probabilità teorica è

$$2^{-16} \approx 1.5259 \cdot 10^{-5},$$

il numero medio atteso di right pairs è

$$10^6 \cdot 2^{-16} \approx 15.26.$$

L'esperimento empirico restituisce un numero di right pairs compatibile con questo valore. Il numero di coppie che raggiungono soltanto la differenza finale è invece maggiore, mostrando concretamente la differenza tra una singola characteristic probability e una differential probability complessiva, che può includere il contributo di più trail.

6 Key recovery

L'ultima parte del progetto utilizza concretamente la caratteristica differenziale per realizzare un recupero parziale della sottochiave. L'attacco è stato costruito su quattro round: i primi tre round vengono utilizzati come distinguisher, mentre il quarto round viene aggiunto come round di key recovery.

La scelta dei tre round iniziali è motivata dalla loro probabilità cumulativa

$$P_3 = 2^{-16},$$

che rende ancora possibile ottenere un numero utile di right pairs con qualche milione di chosen plaintext pairs. Utilizzare un prefisso più lungo renderebbe invece l'esperimento troppo costoso dal punto di vista computazionale.

Il distinguisher fornisce due informazioni fondamentali al key recovery. La prima è la differenza iniziale con cui vengono costruite tutte le coppie:

$$P' = P \oplus (40000000, 0020A008).$$

La seconda è la differenza prevista all'ingresso del quarto round. Se la coppia ha seguito correttamente i primi tre round, allora deve trovarsi nello stato

$$\Delta X_0 = 00020000, \quad \Delta X_1 = 00001000.$$

Da questo stato è possibile ricavare la differenza all'ingresso della S-box del ramo destro del quarto round:

$$\Delta U_1 = \Delta X_1 \oplus \text{ROL}_5(\Delta X_0).$$

Poiché

$$\text{ROL}_5(00020000) = 00400000,$$

si ottiene

$$\Delta U_1 = 00001000 \oplus 00400000 = 00401000.$$

Questo valore rappresenta il target differenziale utilizzato nella fase di key guessing.

La differenza 00401000 contiene soltanto due nibble attivi. Questo permette di ridurre drasticamente lo spazio di ricerca: invece di provare tutti i 2^{32} possibili valori della parola K_1 del quarto round, è sufficiente indovinare gli 8 bit corrispondenti ai due nibble interessati. Il numero totale di ipotesi è quindi

$$2^8 = 256.$$

Prima dell'enumerazione delle chiavi viene applicato un filtro sulle coppie di ciphertext. Dalla caratteristica si conosce la struttura dei nibble che possono risultare attivi nel ramo destro dopo il quarto round. Viene quindi costruita una maschera del tipo

$$00F0F000,$$

che seleziona le sole posizioni ammesse. Per ogni coppia di ciphertext viene calcolata la differenza sul word interessato e si verifica che non siano presenti bit attivi al di fuori dei nibble selezionati. Le coppie che non rispettano questa condizione vengono scartate, mentre quelle compatibili vengono raccolte mediante **Reap** e **Sow**. Questo filtro non garantisce che una coppia sia una right pair, ma riduce significativamente il numero di candidati da sottoporre alla fase successiva.

Per ciascuna delle 256 ipotesi di chiave vengono estratti i due nibble candidati e posizionati correttamente all'interno di una parola parziale **partialKey**. Su ogni coppia candidata viene quindi effettuata una decifratura parziale dell'ultimo round. Dal momento che il ramo destro del ciphertext può essere descritto schematicamente come

$$C_1 = K_1 \oplus S(U_1),$$

si prova a rimuovere l'effetto della chiave mediante

$$C_1 \oplus K_1^*,$$

dove K_1^* è la key guess corrente. Applicando quindi la S-box inversa, che coincide con la S-box stessa, si ottiene una stima di U_1 . La differenza ricostruita tra le due cifrature è quindi

$$\Delta U_1^* = U_1^* \oplus U_1'^*.$$

Se nei due nibble attaccati questa differenza coincide con il target

$$00401000,$$

il contatore associato alla key guess viene incrementato. Le chiavi errate possono produrre alcuni match casuali, ma la chiave corretta è compatibile con le right pairs prodotte dal distinguisher e accumula quindi un numero di hit significativamente maggiore.

Alla fine dell'enumerazione, le 256 ipotesi vengono ordinate in base al numero di hit. Nell'esecuzione finale del progetto il candidato corretto emerge con un picco netto rispetto agli altri valori. La parte reale della sottochiave e la miglior guess coincidono, confermando il recupero corretto di due nibble, cioè 8 bit, della parola K_1 del quarto round.

7 Conclusioni

Il progetto mostra in modo completo il passaggio dalla struttura interna di un cifrario lightweight a un attacco differenziale pratico. L'implementazione di LELBC ha permesso di verificare il comportamento della cifratura e del key schedule; la costruzione della DDT ha fornito le probabilità delle transizioni della S-box; queste probabilità sono state utilizzate per ricostruire e verificare la caratteristica differenziale riportata nel paper; la simulazione Monte Carlo ha confermato sperimentalmente la probabilità del prefisso a tre round; infine, la stessa caratteristica è stata utilizzata come distinguisher per guidare un recupero parziale di chiave.

DDT, caratteristica, distinguisher e key recovery non sono fasi indipendenti: la DDT permette di quantificare le transizioni differenziali della S-box; le transizioni costruiscono la probabilità della caratteristica; la caratteristica fornisce sia la differenza iniziale da imporre alle coppie sia la differenza interna attesa dopo i primi round; questa differenza interna diventa infine il vincolo statistico con cui discriminare le ipotesi sulla sottochiave. Il recupero dei nibble corretti di K_1 mostra quindi in modo concreto come l'informazione differenziale possa essere trasformata da semplice distinguisher a strumento di key recovery.